NEP Aligned

1ICPC317

AY 2025-26

SDLC Laboratory

Quality Laboratory Manual

Experiment No. 03

To perform the system analysis: Requirement analysis, SRS

Course Instructor –
Dr. Tahseen A. Mulla and Prof. Amruta R. Patil

Experiment No. 03

Title of Experiment: To perform the system analysis: Requirement analysis, SRS

Aim of Experiment: To perform system analysis by identifying software requirements and preparing Software Requirement Specification (SRS) document for a given problem domain

System Requirements –

- Operating System: Windows 10 or above
- RAM: Minimum 4 GB
- Processor: 2.33 GHz or higher

Software/s Needed for Experiment

- PDF viewer/ Office suite (MS Word/ LibreOffice)
- Web Browser (for reference)

Experiment Objectives:

- To understand the concept of system analysis
- To perform requirement analysis for a real-world problem
- To differentiate functional and non-functional requirements
- To prepare a basic SRS document
- To understand the structure and importance of SRS

Experiment Outcomes:

After completing the experiment, students will be able to –

- Analyze a system problem and identify requirements
- Categorize requirements into functional and non-functional
- Prepare a structured SRS document
- Understand how SRS guides design and development
- Apply requirement analysis concepts to practical projects

Theory:

What is Requirement Analysis?

Requirement analysis is a foundational step in software development that focuses on identifying and documenting the needs of stakeholders. It serves as the blueprint for the entire development process, ensuring that the final software product meets user expectations and business goals

During this phase, both functional and non-functional requirements are collected through communication with clients, users, and other stakeholders. A thorough and clear requirement analysis minimizes the risks of miscommunication, reduces costly changes later in the project, and sets the stage for efficient design and development

Requirement analysis is a crucial stage in software development where the needs and expectations of stakeholders are identified and documented. This phase ensures that the development team clearly

understands what the software should achieve and the specific conditions it must meet. Thoroughly gathering and analyzing requirements at the start helps avoid misunderstandings and reduces the likelihood of costly changes later in development.

The requirements are typically categorized into two types: functional requirements and non-functional requirements –

- **Functional Requirements**: These define the specific actions the software must be able to perform. Functional requirements focus on the core features and operations that the system needs to support
  Example: For an online banking application, a functional requirement might be: "The system must allow users to transfer funds between accounts."

- **Non-Functional Requirements**: Unlike functional requirements, non-functional requirements address the quality and performance of the system. They include criteria such as speed, security, scalability, and user experience, and describe how the system should perform under various conditions
  Example: A non-functional requirement for the same banking system might be: "The application should be able to handle 1,000 transactions per minute without performance issues."

Both functional and non-functional requirements are vital for ensuring that the software not only fulfills its intended tasks but also performs efficiently and meets user expectations. Properly defining these requirements upfront helps guide the development process, leading to a more successful project outcome

Importance of Requirement Analysis in Software Development
Requirement analysis plays a critical role in the success of any software development project. It sets the foundation for the entire development process by ensuring that both the development team and stakeholders have a clear and shared understanding of what needs to be built. Without a thorough requirement analysis, projects are at risk of delays, miscommunication, and cost overruns.

Here's why requirement analysis is so important:

- **Clear Understanding of Stakeholder Needs**: Through effective requirement analysis, developers gain a detailed understanding of the needs, expectations, and goals of all stakeholders, including clients, end-users, and business leaders. This helps ensure that the final product aligns with user needs and business objectives.
- **Avoiding Scope Creep**: Properly defined requirements help prevent "scope creep," which refers to uncontrolled changes or additions to the project's scope. By clearly documenting the requirements at the beginning, it becomes easier to manage any changes or adjustments, ensuring they are made within the project's boundaries.
- **Improved Communication and Collaboration**: Requirement analysis encourages ongoing communication between developers, project managers, and stakeholders. This collaborative approach ensures that all parties are on the same page and can address potential misunderstandings before they become problems.
- **Reduced Development Costs**: When requirements are clear and well-documented from the start, the development team can focus on delivering the right solution without needing frequent revisions. This helps minimize the time spent on rework and cuts down overall development costs.

- **Minimized Risks and Errors**: Identifying and documenting requirements early helps to reduce the risks of technical errors and misalignments. A solid requirement analysis acts as a roadmap for the development team, helping them avoid building the wrong features or delivering incomplete solutions.
- **Better Project Planning**: Requirement analysis provides the necessary input for creating detailed project timelines, resource allocation plans, and budget estimates. With clear requirements in hand, project managers can plan more accurately, ensuring that the project stays on track and within budget.
- **Improved Quality and User Satisfaction**: By gathering functional and non-functional requirements carefully, the final product is more likely to meet both the technical specifications and user expectations. This leads to higher product quality and greater user satisfaction.

Steps involved in the Requirement Analysis Process

The requirement analysis process ensures everyone understands exactly what needs to be built before development starts. It helps set clear expectations and ensures the app delivers what users want and what the business needs.

Here are the key steps involved using the example of developing a mobile shopping app:

- **Step 1: Identifying Stakeholders and Communicating Needs**
  - The first thing you need to do is identify the key stakeholders. These are the people who will be impacted by the app, such as business owners, users, and the marketing team. Once identified, having clear conversations with them is important to gather their needs and expectations
  - Example: For a shopping app, stakeholders might include the business owners (who want the app to be profitable), the customers (who want an easy shopping experience), and the marketing team (who want user data for promotions)
- **Step 2: Gathering Requirements**
  - Next, you'll start collecting all the necessary details about what the app should do. You'll talk to users and business leaders and look at competitors to determine the must-have features. Don't forget the performance requirements, like how fast the app should be or how secure it needs to be
  - Example: After talking to users, you find out that they really want an easy way to search for products, add them to a shopping cart, and check out quickly. You also learn that fast load times and secure payment methods are critical
- **Step 3: Analyzing and Prioritizing Requirements**
  - Once you have all the information, it's time to analyze what's feasible and prioritize what needs to be done first. Some features might be critical to launch, while others can wait for later updates
  - Example: You decide that the app must include features like user authentication and payment processing from day one, while features like user profiles and loyalty programs can come in later releases
- **Step 4: Documenting Requirements**
  - With the prioritized list of features, it's time to write them down clearly in a formal document. This is like your blueprint for the app, making sure everything is captured in one place
  - Example: You create a detailed Requirements Document outlining key features like browsing products by category, adding items to the cart, and securely checking out. You

also include non-functional requirements like the app needing to handle up to 5,000 users at a time

- **Step 5: Validating Requirements**
  - Once the requirements are written down, it's crucial to go over them with the stakeholders to make sure everything matches their needs. This helps catch any misunderstandings early
  - Example: You share the document with the business owner to confirm that the features, such as a user-friendly search bar and a simple checkout process, align with their business vision for the app
- **Step 6: Creating Use Cases or User Stories**
  - Now, you create user stories or use cases. These are short descriptions of how users will interact with the app, which will guide the design and development process
  - Example: You write a user story like: "As a user, I want to filter products by category so that I can quickly find what I'm looking for," or "As a user, I want to pay using a credit card so that I can complete my purchase securely."
- **Step 7: Getting Sign-off and Approval**
  - After validating the requirements, you'll get the formal sign-off from stakeholders. This is a green light for development to begin, knowing that everyone's on the same page
  - Example: After reviewing the requirements document, the client approves the features for the app, like login, product search, and payment processing, and you're ready to start development
- **Step 8: Managing Changes**
  - Throughout the development process, you may encounter new insights or feedback that require changes to the requirements. It's important to have a plan for how to handle these changes without affecting the project's scope or timeline
  - Example: After launching the app, users request a "wishlist" feature. You review this feedback with stakeholders, and once it's approved, you update the requirements to include this feature in the next version

By following these steps, the requirement analysis process ensures that the mobile shopping app is aligned with what users and the business need, helping the development team stay focused and efficient

What is Software Requirements Specification (SRS)?

This is a comprehensive and detailed description of the intended purpose and environment for the system under development. The solution architecture is presented in a structured format with functional and non-functional requirements, system constraints, assumptions, and dependencies. SRS contains what the software needs to achieve for all stakeholders, such as business analysts, developers, QA teams, project managers, end-users, etc

Therefore, SRS is a binding contract to minimize differences between the functional specification and the final product. It basically aligns the entire project. The project follows the design closely throughout the chain (after requirements, through implementation, testing, and deployment), and it is sometimes referred to as a single source of truth

Why is an SRS document important?

It is so important that you will see how it sets the stage for your entire project, helping keep everyone clear, aligned, and accurate at all stages of development

Below are the key reasons why an SRS document is important:

- **Provides Clarity**: Defines what the software will and what it will not do. So, leaves no room for ambiguity. This ensures that all stakeholders have the same understanding of the project. Through these clear expectations, misunderstandings and misaligned deliverables are avoided.
- **Brings in Communication**: Acting as a shared reference point, the SRS bridges the gap between technical and non-technical stakeholders. It translates business objectives into actionable technical requirements, ensuring everyone speaks the same language.
- **Roadmap for Development and Testing**: Outlines the features and functionalities to developers that they need to implement. And for QA teams, it provides a baseline for test case design and to check whether the software meets its requirements. This structured approach ensures development and testing efforts align with project goals.
- **Removes Risks**: By identifying requirements, constraints, and potential challenges upfront, the SRS minimizes surprises during development. You can address risks early to reduce the cost of rework. This approach saves time, effort, and resources throughout the SDLC.
- **Follows Compliance**: SRS document is often required to demonstrate adherence to standards and laws. It helps capture compliance-related requirements. This helps to make sure that the software meets industry and legal guidelines. So this reduces the risk of non-compliance penalties and enhances credibility with regulators

Components of an SRS Document

- **Introduction**
  - Purpose
  - Scope
  - Definition, acronyms and abbreviations
  - References
- **System overview**
  - Systems goals
  - Aims to solve
  - Key functions and its role
- **Functional requirements**
  - Features
  - User scenario
  - Data requirements
- **Non-functional requirements**
  - Performance
  - Usability
  - Reliability
  - Security
  - Scalability
  - Maintainability
- **System constraints**
  - Technological constraints
  - Budgetary constraints
  - Regulatory constraints
  - Time constraints
- **External interface requirements**
  - User interfaces
  - Hardware interfaces

  - Software interfaces
  - Communication interfaces
- **Assumptions and dependencies**
  - Assumptions about user behavior or system environment
  - Dependencies on third-party libraries, external APIs or specific hardware
- **Appendices**
  - Diagrams and charts
  - Glossary
  - Detailed data model

How to write and SRS document?

Task to Perform

Case Study: Student Feedback Management System

An engineering institute plans to develop a Student Feedback Management System to collect feedback from student about courses, faculty and infrastructure, generate analytical reports and support continuous improvement

Task for students

- Perform system analysis and prepare a basic SRS for the above case study by completing the following –
  1. **Problem understanding** – Write a brief problem statement (4-5 lines)
  2. **Stakeholder identification** – List key stakeholders (minimum 3)
  3. **Requirement analysis** – Identify at least 5 functional requirements and 3 non-functional requirements

4. **SRS preparation** – Prepare an SRS document which consists of: Introduction, Overall description, System features, External Interfaces, Non-functional requirements and Assumptions and constraints
5. **Validation** – Review the SRS for clarity, completeness and consistency

Submission Requirements –

Students must submit a single PDF or neatly written lab record containing –

- Problem statement/ case study summary
- List of stakeholders
- Functional requirements (minimum 5)
- Non-functional requirements (minimum 3)
- SRS document with standard headings
- Mention the assumptions and constraints

Observations:

- Requirement analysis clarifies system scope
- Proper documentation avoids ambiguity
- SRS acts as a reference throughout SDLC
- Clear requirements reduce development errors

Conclusion:

This experiment demonstrated how system analysis and requirement analysis lead to the creation of an SRS document. A well-prepared SRS forms the foundation for successful software design, implementation and testing

Expected Oral Questions:

1) What is system analysis?
2) What is requirement analysis?
3) Define SRS
4) Differentiate functional and non-functional requirements
5) Why is SRS important?
6) List the main components of SRS
7) What problems arise due to poor requirement analysis?

FAQs in Interview:

1) What is SRS?
2) Who prepares the SRS?
3) Can SRS change during development?
4) Why is SRS called a contract?
5) What happens if SRS is incomplete?